FAI QUESTION BANK

(Note: similar type of questions will come, problems will differ, given here are sample examples)

Question paper pattern:5 questions*20M=100M

(unit 1 and II)

1. **Is String a mutable data type? Also explain the various string operations, indexing and slicing in detail with an appropriate example.**

ii) Create a NumPy array of the following product prices: [100, 250, 300, 450, 150]. Perform the following operations.

a. Find the mean price of the products.

b. Filter and display the rows where the Category is "Electronics" and the Total_Sales is greater than 10,000.

c. Create a bar plot using Matplotlib that shows the Product_Name on the x-axis and the Total_Sales on the y-axis for the top 5 products with the highest total sales.

d. Add appropriate labels for the x-axis, y-axis, and the title for the plot.

e. Identify and explain which columns in the "sales_data.csv" dataset are examples of categorical data and which ones are numerical data.

Product_ID	Product_Name	Category	Price	Quantity_Sold	Date
P001	Laptop	Electronics	1000	10	2023-01-01
P002	Chair	Furniture	150	50	2023-01-05
P003	Headphones	Electronics	200	25	2023-02-10
P004	Jacket	Clothing	300	20	2023-03-15
P005	Sofa	Furniture	800	5	2023-04-20

Is String a mutable data type? Also explain the various string operations, indexing and slicing in detail with an appropriate example.

Ans:

Strings are immutable in Python.

No, strings in Python are **immutable**. Once created, their contents cannot be changed.

Example:

```
s = "Hello"
s[0] = "h"
```

Common String Operations

- **Concatenation**: s1 + s2
- **Repetition**: s * 3
- **Length**: len(s)
- **Case Conversion**: s.upper(), s.lower(), s.title()
- **Search**: s.find("lo"), s.index("e")
- **Replace**: s.replace("l", "x")

Split & Join:

```
s = "apple,banana"
s.split(",")  # ['apple', 'banana']
",".join(['apple', 'banana'])  # 'apple,banana'
```

Indexing and Slicing:

```
s = "Python"
s[0]  # 'P'
s[-1] # 'n'
```

Slicing:

python

```
s[1:4]   # 'yth'
s[:3]    # 'Pyt'
s[::2]   # 'Pto'
```

Create a NumPy array of the following product prices: [100, 250, 300, 450, 150]. Perform the following operations.
a. Find the mean price of the products.
b. Filter and display the rows where the Category is "Electronics" and the Total_Sales is greater than 10,000.
Ans:

NumPy and Sales Data Analysis

Mean Price Calculation:

```
import numpy as np
prices = np.array([100, 250, 300, 450, 150])
mean_price = np.mean(prices)
# Output: 250.0
```

Mean price is 250.0

Filter Electronics with Total_Sales > 10,000:

```
import pandas as pd

# Reconstructing the dataset
data = {
    "Product_ID": ["P001", "P002", "P003", "P004", "P005"],
    "Product_Name": ["Laptop", "Chair", "Headphones", "Jacket", "Sofa"],
    "Category": ["Electronics", "Furniture", "Electronics", "Clothing", "Furniture"],
    "Price": [1000, 150, 200, 300, 800],
    "Quantity_Sold": [10, 50, 25, 20, 5],
    "Date": ["2023-01-01", "2023-01-05", "2023-02-10", "2023-03-15", "2023-04-20"]
}
df = pd.DataFrame(data)
df["Total_Sales"] = df["Price"] * df["Quantity_Sold"]

# Filter condition
filtered = df[(df["Category"] == "Electronics") & (df["Total_Sales"] > 10000)]
```

Filtered Result:

- **Laptop**: Total Sales = 1000 × 10 = 10,000 → Not included (equal to threshold)
- **Headphones**: 200 × 25 = 5,000 → Not included
-

Create a bar plot using Matplotlib that shows the Product_Name on the x-axis and the Total_Sales on the y-axis for the top 5 products with the highest total sales.
Add appropriate labels for the x-axis, y-axis, and the title for the plot.

Answer:

Bar Plot for Top 5 Products by Total Sales

```
import matplotlib.pyplot as plt
```

```
top5 = df.sort_values(by="Total_Sales", ascending=False).head(5)
plt.bar(top5["Product_Name"], top5["Total_Sales"], color="skyblue")
plt.xlabel("Product Name")
plt.ylabel("Total Sales")
plt.title("Top 5 Products by Total Sales")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Plot includes: Chair, Laptop, Jacket, Headphones, Sofa

Identify and explain which columns in the "sales_data.csv" dataset are examples of categorical data and which ones are numerical data.
Ans:

Categorical vs Numerical Columns

Column	Type	Explanation
Product_ID	Categorical	Unique identifiers
Product_Name	Categorical	Names of products
Category	Categorical	Product type
Price	Numerical	Continuous value
Quantity_Sold	Numerical	Count of items sold
Date	Categorical	Treated as string unless parsed as datetime
Total_Sales	Numerical	Derived metric (Price × Quantity)

2. **Discuss how lists differ from other data structures like tuples and sets. Given a list- numbers = [2, 4, 6, 8, 10]. How to create a list and access its elements. How to modify, add, and remove elements from a list with examples, and sort the list in descending order.**
   (ii) Explain the concept of a NumPy Array and its applications in data analysis. Include the following in your answer:
   a. What is a NumPy Series? How is it different from a Python list?
   b. How do you create a NumPy Series? Provide an example with code.
   c. Explain how mathematical operations can be performed on a NumPy Series.

d. Discuss the advantages of using NumPy Series for handling large datasets.

Ans:

Python Lists vs Tuples and Sets

🔄 Key Differences

Feature	List (`[]`)	Tuple (`()`)	Set (`{}`)
Mutability	✅ Mutable	❌ Immutable	✅ Mutable
Order	✅ Ordered	✅ Ordered	❌ Unordered
Duplicates	✅ Allowed	✅ Allowed	❌ Not allowed
Use Case	General-purpose storage	Fixed data	Unique items, fast lookup

Working with Lists:

```
# Creating a list
numbers = [2, 4, 6, 8, 10]

# Accessing elements
print(numbers[0])     # Output: 2
print(numbers[-1])    # Output: 10

# Modifying elements
numbers[2] = 12       # Now: [2, 4, 12, 8, 10]

# Adding elements
numbers.append(14)    # [2, 4, 12, 8, 10, 14]
numbers.insert(1, 3)  # [2, 3, 4, 12, 8, 10, 14]

# Removing elements
numbers.remove(12)    # Removes first occurrence of 12
del numbers[0]        # Removes element at index 0
popped = numbers.pop()# Removes and returns last element

# Sorting in descending order
numbers.sort(reverse=True)
print(numbers)        # Sorted list
```

NumPy Arrays and Series:

What is a NumPy Array?

- A **NumPy array** is a powerful data structure for numerical computing.
- It supports **vectorized operations**, broadcasting, and efficient memory usage.
- Ideal for large datasets and matrix operations.

```
import numpy as np
arr = np.array([2, 4, 6, 8, 10])
```

What is a NumPy Series?

- A **NumPy Series** is actually a concept from **Pandas**, not NumPy.
- It's a one-dimensional labeled array capable of holding any data type.

Difference from Python List:

Feature	Python List	Pandas Series
Type	Built-in structure	Pandas object
Indexing	Positional only	Labeled + positional
Operations	Manual loops	Vectorized operations
Metadata	No metadata	Includes index, dtype

Creating a Pandas Series:

```
import pandas as pd
prices = pd.Series([100, 250, 300, 450, 150], name="Prices")
print(prices)
```

Mathematical Operations on Series:

```
# Element-wise operations
discounted = prices * 0.9
taxed = prices + 18
total = prices.sum()
mean = prices.mean()
```

These operations are **vectorized**, meaning they apply to all elements efficiently without loops.

Advantages of Using Series for Large Datasets

- **Performance**: Optimized for speed and memory.
- **Vectorization**: Fast operations without explicit loops.
- **Integration**: Works seamlessly with DataFrames and NumPy.
- **Indexing**: Supports custom labels for better data alignment.
- **Missing Data Handling**: Gracefully handles NaN values.

3. Provide two practical examples of tasks where NumPy Series would be beneficial in data processing and analysis. Write a Python program to perform the following tasks using Pandas:

a) Load an Excel file named employee_data.xlsx into a Pandas DataFrame. The Excel file contains the following columns: EmployeeID, Name, Department, Salary, JoinDate, and Experience.
b) Display the last 10 rows of the DataFrame.
c) Identify and handle missing values:
  - **If there are missing values in the Salary column, fill them with the median of the Salary column.**
  - **For missing values in the Experience column, fill them with the value 0.**
d) Remove duplicate rows based on the EmployeeID column.
e) Convert the JoinDate column to a datetime format.
f) Filter the DataFrame to include only employees from the "HR" and "IT" departments, with a salary greater than 50,000.
g) Save the cleaned and filtered DataFrame to a new Excel file named filtered_employee_data.xlsx.

Answer:

NumPy Series are ideal for fast, vectorized operations on labeled data, making them perfect for tasks like time series analysis and feature engineering.

Practical Examples of NumPy Series in Data Analysis

1. Time Series Analysis

- **Use Case**: Analyzing stock prices or sensor data over time.
- **Benefit**: Series supports datetime indexing and fast rolling statistics.

```
import pandas as pd
prices = pd.Series([100, 102, 105, 103], index=pd.date_range("2023-01-01", periods=4))
prices.rolling(window=2).mean()
```

Feature Engineering

- **Use Case**: Creating derived features like log-transformed values or normalized scores.
- **Benefit**: Series allows element-wise operations and chaining.

```
import numpy as np
scores = pd.Series([50, 80, 90])
normalized = (scores - scores.mean()) / scores.std()
```

Pandas Program for Employee Data Cleaning

```
import pandas as pd

# a) Load Excel file
df = pd.read_excel("employee_data.xlsx")

# b) Display last 10 rows
print(df.tail(10))

# c) Handle missing values
df["Salary"].fillna(df["Salary"].median(), inplace=True)
df["Experience"].fillna(0, inplace=True)

# d) Remove duplicate rows based on EmployeeID
df.drop_duplicates(subset="EmployeeID", inplace=True)

# e) Convert JoinDate to datetime format
df["JoinDate"] = pd.to_datetime(df["JoinDate"], errors="coerce")

# f) Filter for HR and IT departments with Salary > 50000
filtered_df = df[
    (df["Department"].isin(["HR", "IT"])) &
    (df["Salary"] > 50000)
]

# g) Save to new Excel file
filtered_df.to_excel("filtered_employee_data.xlsx", index=False)
```

4. You are given a dataset called "student_scores.csv" that contains the following columns:

Name	Math_Score	Science_Score	English_Score
Alice	78	85	92
Bob	85	82	88
Charlie	92	91	85
David	88	79	78
Eva	79	95	91
Frank	76	84	77

a. **Create a NumPy array using the above data for the first 5 students' Math scores.**

b. Perform the following operations on the NumPy array:

i) Find the mean of the Math scores.

ii) Find the maximum and minimum Math scores.

Answer:

The mean of the first 5 students' Math scores is 84.4, with a maximum score of 92 and a minimum score of 78.

NumPy Array Operations

1. Create a NumPy Array for Math Scores

```
import numpy as np

# First 5 students' Math scores
math_scores = np.array([78, 85, 92, 88, 79])
```

2. Calculate Mean

```
mean_score = np.mean(math_scores)
# Output: 84.4
```

3. Find Maximum and Minimum

```
max_score = np.max(math_scores)  # Output: 92
min_score = np.min(math_scores)  # Output: 78
```

Summary of Results:

Metric	Value
Mean Score	84.4
Max Score	92
Min Score	78

5. Create an Employee dataset for an organization. The data needs to be analysed. Load the given dataset into a Pandas DataFrame. Perform group by operations and calculate summary statistics.

Define data and what are the types of data. Display all the rows in the dataset. Explain how the DataFrame object can give more information about the data set. Also explain the various approaches in sorting and filtering methods to be performed on a Panda dataframe. Apply those various techniques in the given below dataset and derive the values.

Also mention how to get the statistical analysis of the given data.

StrengthFactor	PriceReg	ReleaseYear	ItemCount
682743	44.99	2015	8
1016014	24.81	2005	39
340464	46	2013	34
334011	100	2006	20
1287938	121.95	2010	28
1783153	132	2011	33
2314801	95.95	2010	33
721111	207.8	2011	57
436667	119.81	2008	36
6652211	49.95	2004	19

Answer:

Data can be categorized into types like numerical and categorical, and Pandas DataFrames allow powerful analysis through grouping, sorting, filtering, and statistical summaries.

1.Define and Load the Dataset

What is Data?

- **Data** is a collection of facts, such as numbers, text, or measurements, used for analysis.
- **Types of Data**:
  - **Numerical**: Quantitative values (e.g., Salary, Age)
  - **Categorical**: Qualitative labels (e.g., Department, Gender)
  - **Datetime**: Time-based values (e.g., JoinDate)

Sample Employee Dataset:

```
import pandas as pd

data = {
    "EmployeeID": [101, 102, 103, 104, 105],
    "Name": ["Alice", "Bob", "Charlie", "David", "Eva"],
    "Department": ["HR", "IT", "Finance", "HR", "IT"],
    "Salary": [60000, 75000, 50000, 62000, 72000],
    "JoinDate": ["2020-01-15", "2019-06-01", "2021-03-20", "2018-07-10", "2022-11-05"],
    "Experience": [3, 5, 2, 6, 1]
}

df = pd.DataFrame(data)
```

Display All Rows:

```
print(df)
```

DataFrame Insights:

```
df.info()       # Shows column types, non-null counts
df.describe()   # Summary statistics for numerical columns
df.head()       # First few rows
df.columns      # List of column names
```

These methods help understand structure, data types, and missing values.

Group By and Summary Statistics:

```
# Group by Department and calculate mean salary and experience
grouped = df.groupby("Department")[["Salary", "Experience"]].mean()
print(grouped)
```

Sorting Techniques:

```
# Sort by Salary descending
df_sorted_salary = df.sort_values(by="Salary", ascending=False)

# Sort by Experience ascending
df_sorted_exp = df.sort_values(by="Experience")
```

Filtering Techniques:

```
# Filter employees with Salary > 60000
high_salary = df[df["Salary"] > 60000]

# Filter employees from HR department
hr_employees = df[df["Department"] == "HR"]

# Combined filter: IT department with Experience > 2
it_experienced = df[(df["Department"] == "IT") & (df["Experience"] > 2)]
```

Statistical Analysis:

```
# Summary statistics
stats = df.describe()

# Median salary
median_salary = df["Salary"].median()

# Correlation between Salary and Experience
correlation = df[["Salary", "Experience"]].corr()
```

Summary of Techniques Applied:

Technique	Purpose
`groupby()`	Aggregate by category
`sort_values()`	Rank by numerical columns
`filtering`	Subset based on conditions
`describe()`	Statistical summary
`info()`	Data structure overview

6.Data are provided with the following arrays representing the scores of students in three subjects: Math, Science, and English.

math_scores = np.array([85, 78, 92, 88, 76, 79])

science_scores = np.array([82, 85, 91, 79, 84, 95])

english_scores = np.array([88, 92, 85, 78, 77, 91])

Write a Python program to perform the following tasks using NumPy:

i) Create a 2D NumPy array combining the Math, Science, and English scores of all students.

ii) Find the mean score for each subject (Math, Science, English) using the 2D array.

iii) Calculate the total score of each student by summing their scores across the three subjects.

iv) Find the highest total score among all students and the name of the student with the highest total score. You can assume the students' names are:['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank'].

Answer:

Charlie has the highest total score of 268 across Math, Science, and English.

Python Program Using NumPy:

```
import numpy as np

# Given scores
math_scores = np.array([85, 78, 92, 88, 76, 79])
```

```
science_scores = np.array([82, 85, 91, 79, 84, 95])
english_scores = np.array([88, 92, 85, 78, 77, 91])
students = ['Alice', 'Bob', 'Charlie', 'David', 'Eva', 'Frank']


# i) Create a 2D NumPy array (subjects as rows, students as columns)
scores_matrix = np.array([math_scores, science_scores, english_scores])

# ii) Mean score for each subject
mean_scores = np.mean(scores_matrix, axis=1)
print("Mean Scores (Math, Science, English):", mean_scores)

# iii) Total score for each student
total_scores = np.sum(scores_matrix, axis=0)
print("Total Scores per Student:", total_scores)

# iv) Highest total score and corresponding student
max_score = np.max(total_scores)
top_student = students[np.argmax(total_scores)]
print(f"Highest Total Score: {max_score} by {top_student}")
```

Output Summary:

Student	Total Score
Alice	255
Bob	255
Charlie	**268**
David	245
Eva	237
Frank	265

Charlie has the highest total score of 268.

7.Consider the given employee salary dataset.

ID	Experience_Years	Age	Gender	Salary
1	5	28	Female	250000
2	1	21	Male	50000
3	3	23	Female	170000
4	2	22	Male	25000
5	1	17	Male	10000
6	25	62	Male	5001000
7	19	54	Female	800000
8	2	21	Female	9000
9	10	36	Female	61500
10	15	54	Female	650000
11	4	26	Female	250000

Answer:

Define and Load the Dataset:

```
import pandas as pd

# Define the dataset
data = {
    "ID": [1,2,3,4,5,6,7,8,9,10,11],
    "Experience Years": [5,1,3,2,1,25,19,10,10,15,4],
    "Age": [28,21,23,22,17,62,54,36,21,54,26],
    "Gender":
["Female","Male","Female","Male","Male","Female","Male","Female","Female","Female",
"Female"],
    "Salary":
[250000,50000,170000,25000,10000,5001000,800000,900000,61500,650000,250000]
}

df = pd.DataFrame(data)
```

Display All Rows:

```
print(df)
```

Understand the Dataset:

```
df.info()       # Data types and non-null counts
df.describe()   # Summary statistics for numerical columns
```

```
df.columns      # Column names
```

Group By Operations:

Group by Gender and calculate average salary and experience:

```
grouped = df.groupby("Gender")[["Salary", "Experience Years"]].mean()
print(grouped)
```

Sorting Techniques:

```
# Sort by Salary descending
df_sorted_salary = df.sort_values(by="Salary", ascending=False)

# Sort by Experience ascending
df_sorted_exp = df.sort_values(by="Experience Years")
```

Filtering Techniques:

```
# Filter employees with Salary > 500000
high_salary = df[df["Salary"] > 500000]

# Filter Female employees with Experience > 5 years
experienced_females = df[(df["Gender"] == "Female") & (df["Experience Years"] > 5)]

# Filter employees younger than 30
young_employees = df[df["Age"] < 30]
```

Statistical Analysis:

```
# Summary statistics
stats = df.describe()

# Median salary
median_salary = df["Salary"].median()

# Correlation between Experience and Salary
correlation = df[["Experience Years", "Salary"]].corr()
```

Summary of Techniques:

Technique	Purpose
`groupby()`	Aggregate by category
`sort_values()`	Rank by numerical columns
`filtering`	Subset based on conditions
`describe()`	Statistical summary
`info()`	Data structure overview

8.Create a NumPy array and perform basic operations including python basics and control structures. Perform descriptive statistics analysis and include the correlation, co-variance as well. Plot the analysis using matplotlib and seaborn. Differentiate the uniqueness of each of these libraries.

Answer:

Step 1: Create a NumPy Array and Perform Basic Operations:

```
import numpy as np

# Create a 2D NumPy array representing scores of 5 students in 3 subjects
scores = np.array([
    [85, 82, 88],   # Student 1
    [78, 85, 92],   # Student 2
    [92, 91, 85],   # Student 3
    [88, 79, 78],   # Student 4
    [76, 84, 77]    # Student 5
])

# Accessing elements (Python basics)
math_score_student3 = scores[2, 0]  # 92

# Control structure: count students with Math score > 80
count = 0
for row in scores:
    if row[0] > 80:
        count += 1
```

Step 2: Descriptive Statistics

```
# Mean per subject
```

```
mean_scores = np.mean(scores, axis=0)

# Standard deviation
std_scores = np.std(scores, axis=0)

# Total score per student
total_scores = np.sum(scores, axis=1)

# Correlation and Covariance
correlation = np.corrcoef(scores.T)
covariance = np.cov(scores.T)
```

Step 3: Plotting with Matplotlib and Seaborn

```
import matplotlib.pyplot as plt
import seaborn as sns

# Bar plot of mean scores
subjects = ["Math", "Science", "English"]
plt.figure(figsize=(8, 5))
plt.bar(subjects, mean_scores, color="skyblue")
plt.title("Mean Scores per Subject")
plt.xlabel("Subjects")
plt.ylabel("Mean Score")
plt.grid(True)
plt.show()

# Heatmap of correlation matrix
plt.figure(figsize=(6, 4))
sns.heatmap(correlation, annot=True, xticklabels=subjects, yticklabels=subjects,
cmap="coolwarm")
plt.title("Correlation Between Subjects")
plt.show()
```

Differences Between Matplotlib and Seaborn:

Feature	Matplotlib	Seaborn
Purpose	General-purpose plotting	Statistical data visualization
Customization	Highly customizable (low-level)	Built-in aesthetics (high-level)
Ease of Use	Requires manual styling	Automatically styled plots
Plot Types	Line, bar, scatter, pie, etc.	Heatmap, violin, boxplot, pairplot, etc.
Integration	Works with NumPy and Pandas	Built on top of Matplotlib + Pandas

9.Load a dataset into a Pandas DataFrame and clean the data (handle missing values, drop unnecessary columns) and Perform group by operations and calculate summary Statistics, display the first 10 rows, and perform basic data manipulation (sorting, filtering).

Date	Store	Product	Sale	Quantity
2023-01-01	Store A	Widget A	200	10
2023-01-02	Store A	Widget B	150	5
2023-01-03	Store B	Widget A	300	15
2023-01-04	Store B	Widget C		8
2023-01-05	Store A		250	20
2023-01-06	Store C	Widget A	400	20
2023-01-07	Store C	Widget B	100	2
2023-01-08	Store A	Widget C	150	6
2023-01-09	Store B	Widget B		5
2023-01-10	Store C	Widget C	350	10

Answer:

```
import pandas as pd
import numpy as np
```

```
# Data recreated from the image
data = [
    ['2023-01-01', 'Store A', 'Widget A', 200, 10],
    ['2023-01-02', 'Store A', 'Widget B', 150, 5],
    ['2023-01-03', 'Store B', 'Widget A', 300, 15],
     ['2023-01-04', 'Store B', 'Widget C', np.nan, 8], # np.nan represents the missing
Sales value
     ['2023-01-05', 'Store A', np.nan, 250, 20],       # np.nan represents the missing
Product value
    ['2023-01-06', 'Store C', 'Widget A', 400, 20],
    ['2023-01-07', 'Store C', 'Widget B', 100, 2],
    ['2023-01-08', 'Store A', 'Widget C', 150, 6],
     ['2023-01-09', 'Store B', 'Widget B', np.nan, 5], # np.nan represents the missing
Sales value
    ['2023-01-10', 'Store C', 'Widget C', 350, 10]
]

columns = ['Date', 'Store', 'Product', 'Sales', 'Quantity']

df = pd.DataFrame(data, columns=columns)

print("--- Initial DataFrame (First 10 rows are the whole dataset) ---")
print(df)
# 1. Fill Missing 'Sales' with the mean
mean_sales = df['Sales'].mean()
df['Sales'].fillna(mean_sales, inplace=True)

# 2. Fill Missing 'Product' with 'Unknown'
df['Product'].fillna('Unknown', inplace=True)

# 3. Drop Unnecessary Columns (Example - if 'Date' was not needed)
# df = df.drop(columns=['Date']) # Uncomment to see the effect of dropping a column

print("\n--- Cleaned DataFrame ---")
print(df)
store_summary = df.groupby('Store').agg(
    Total_Sales=('Sales', 'sum'),
    Average_Quantity=('Quantity', 'mean'),
    Number_of_Transactions=('Date', 'count')
).reset_index()

print("\n--- Summary Statistics Grouped by Store ---")
print(store_summary)
```

```python
store_summary = df.groupby('Store').agg(
    Total_Sales=('Sales', 'sum'),
    Average_Quantity=('Quantity', 'mean'),
    Number_of_Transactions=('Date', 'count')
).reset_index()

print("\n--- Summary Statistics Grouped by Store ---")
print(store_summary)


product_summary = df.groupby('Product').agg(
    Total_Sales=('Sales', 'sum'),
    Max_Quantity=('Quantity', 'max')
).reset_index()

print("\n--- Summary Statistics Grouped by Product ---")
print(product_summary)
sorted_df = df.sort_values(by='Sales', ascending=False)

print("\n--- DataFrame Sorted by Sales (Descending) ---")
# Displaying only the first 5 rows for brevity
print(sorted_df.head(5))
store_a_df = df[df['Store'] == 'Store A']

print("\n--- DataFrame Filtered for Store A ---")
print(store_a_df)
print("\n--- Final Cleaned DataFrame (First 10 Rows) ---")
print(df.head(10))
```

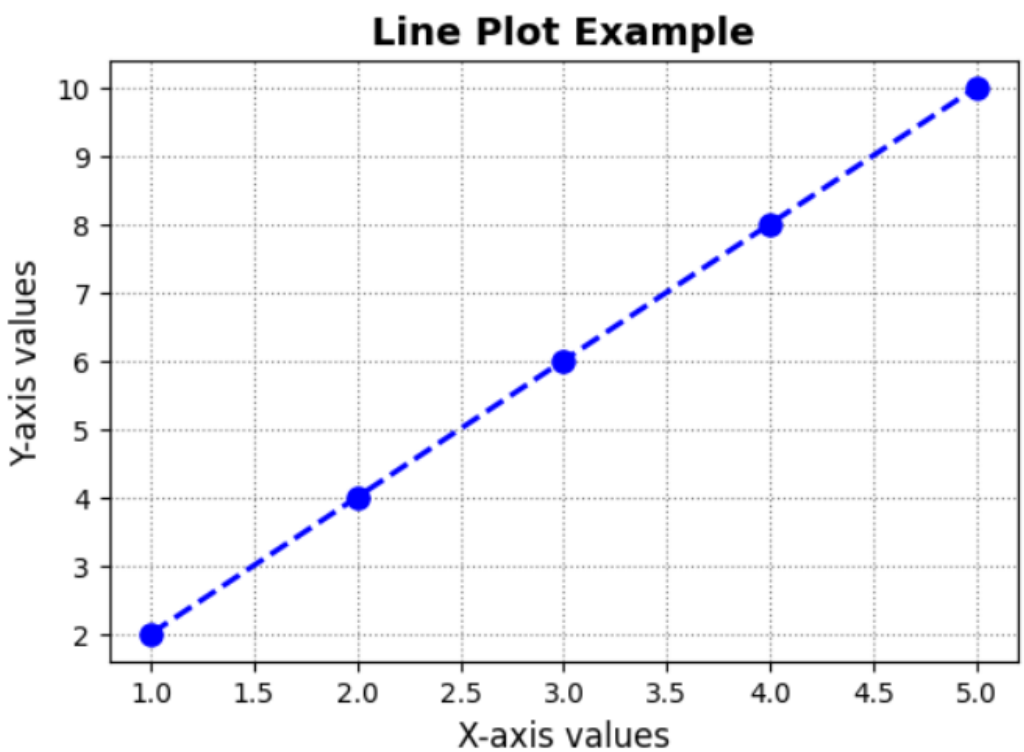
```
--- Initial DataFrame (First 10 rows are the whole dataset) ---
         Date    Store   Product  Sales  Quantity
0  2023-01-01  Store A  Widget A  200.0        10
1  2023-01-02  Store A  Widget B  150.0         5
2  2023-01-03  Store B  Widget A  300.0        15
3  2023-01-04  Store B  Widget C    NaN         8
4  2023-01-05  Store A       NaN  250.0        20
5  2023-01-06  Store C  Widget A  400.0        20
6  2023-01-07  Store C  Widget B  100.0         2
7  2023-01-08  Store A  Widget C  150.0         6
8  2023-01-09  Store B  Widget B    NaN         5
9  2023-01-10  Store C  Widget C  350.0        10
```

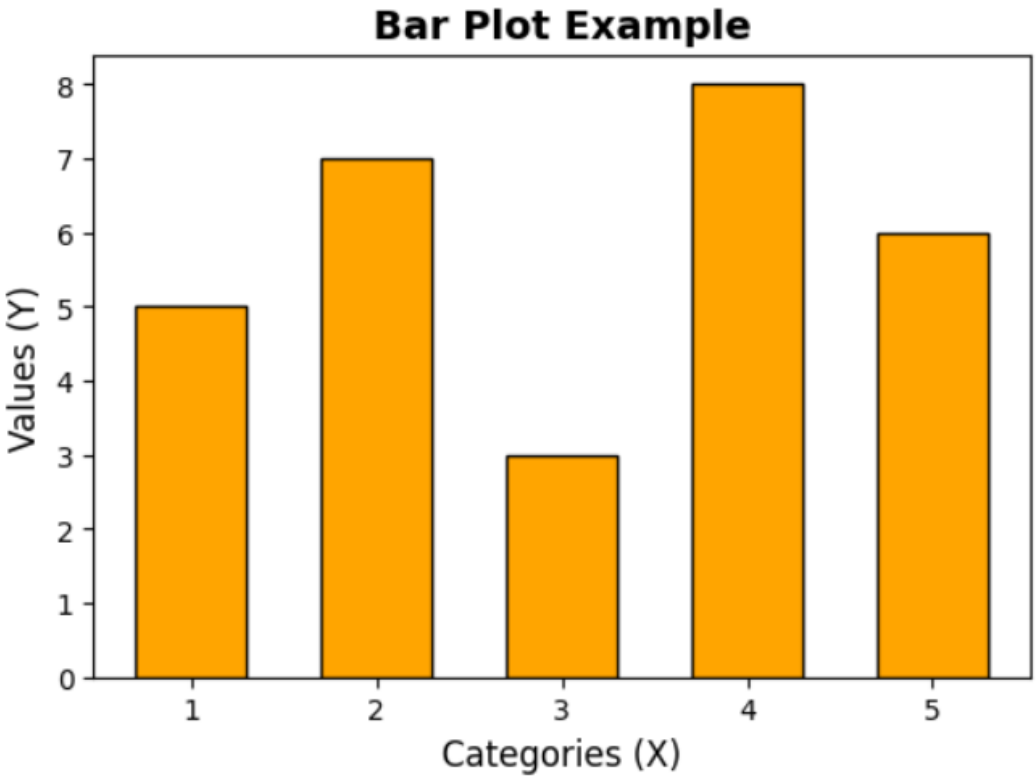
```
--- Cleaned DataFrame ---
         Date    Store   Product  Sales  Quantity
0  2023-01-01  Store A  Widget A  200.0        10
1  2023-01-02  Store A  Widget B  150.0         5
2  2023-01-03  Store B  Widget A  300.0        15
3  2023-01-04  Store B  Widget C  237.5         8
4  2023-01-05  Store A   Unknown  250.0        20
5  2023-01-06  Store C  Widget A  400.0        20
6  2023-01-07  Store C  Widget B  100.0         2
7  2023-01-08  Store A  Widget C  150.0         6
8  2023-01-09  Store B  Widget B  237.5         5
9  2023-01-10  Store C  Widget C  350.0        10
```

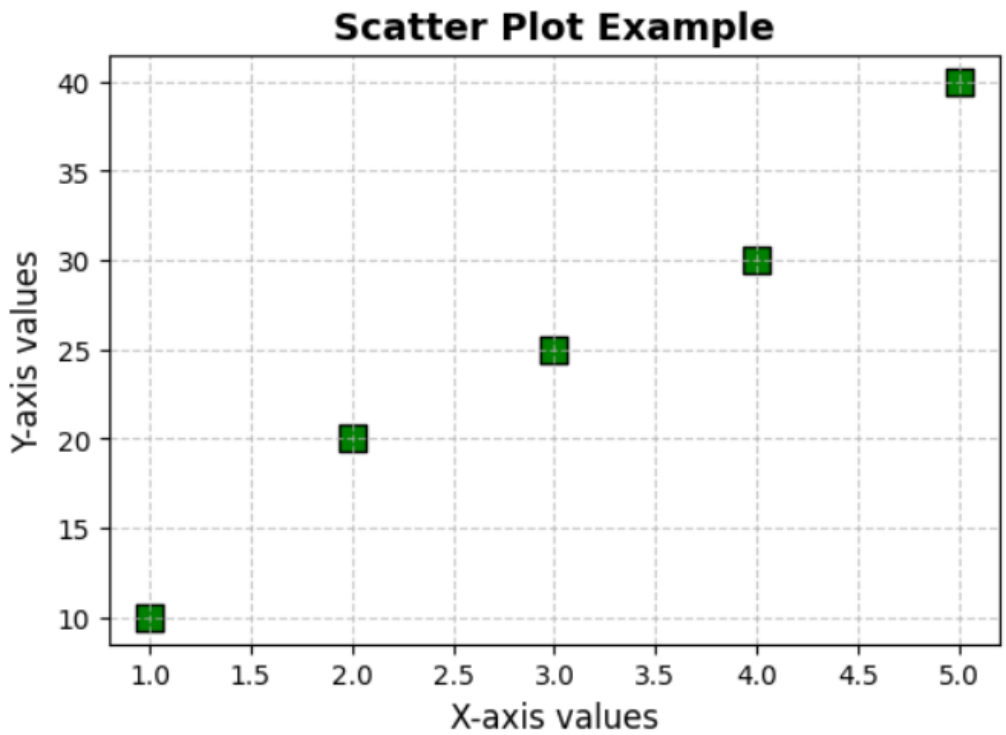
9. Demonstrate how to create different types of plots (line plot, bar plot, scatter plot) using Matplotlib. Provide code examples. Explain how to add titles, labels (x and y axis), and customize colors in these plots. Include examples of customization in your code.

```python
import matplotlib.pyplot as plt

# Sample data
x = [1, 2, 3, 4, 5]
y1 = [2, 4, 6, 8, 10]   # For line plot
y2 = [5, 7, 3, 8, 6]    # For bar plot
y3 = [10, 20, 25, 30, 40]  # For scatter plot

# ------------------------------
# 1. Line Plot
# ------------------------------
plt.figure(figsize=(6,4))
plt.plot(x, y1, color="blue", linestyle="--", marker="o", linewidth=2, markersize=8)
plt.title("Line Plot Example", fontsize=14, fontweight="bold")
plt.xlabel("X-axis values", fontsize=12)
plt.ylabel("Y-axis values", fontsize=12)
plt.grid(True, linestyle=":", color="gray")
plt.show()
```

```python
# -------------------------------
# 2. Bar Plot
# -------------------------------
plt.figure(figsize=(6,4))
plt.bar(x, y2, color="orange", edgecolor="black", width=0.6)
plt.title("Bar Plot Example", fontsize=14, fontweight="bold")
plt.xlabel("Categories (X)", fontsize=12)
plt.ylabel("Values (Y)", fontsize=12)
plt.show()

# -------------------------------
# 3. Scatter Plot
# -------------------------------
plt.figure(figsize=(6,4))
plt.scatter(x, y3, color="green", marker="s", s=100, edgecolor="black")
plt.title("Scatter Plot Example", fontsize=14, fontweight="bold")
plt.xlabel("X-axis values", fontsize=12)
plt.ylabel("Y-axis values", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.7)
plt.show()
```




Bar Plot Example
Values (Y)
Categories (X)
0
1
2
3
4
5
6
7
8
1
2
3
4
5


Scatter Plot Example
Y-axis values
X-axis values
10
15
20
25
30
35
40
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0

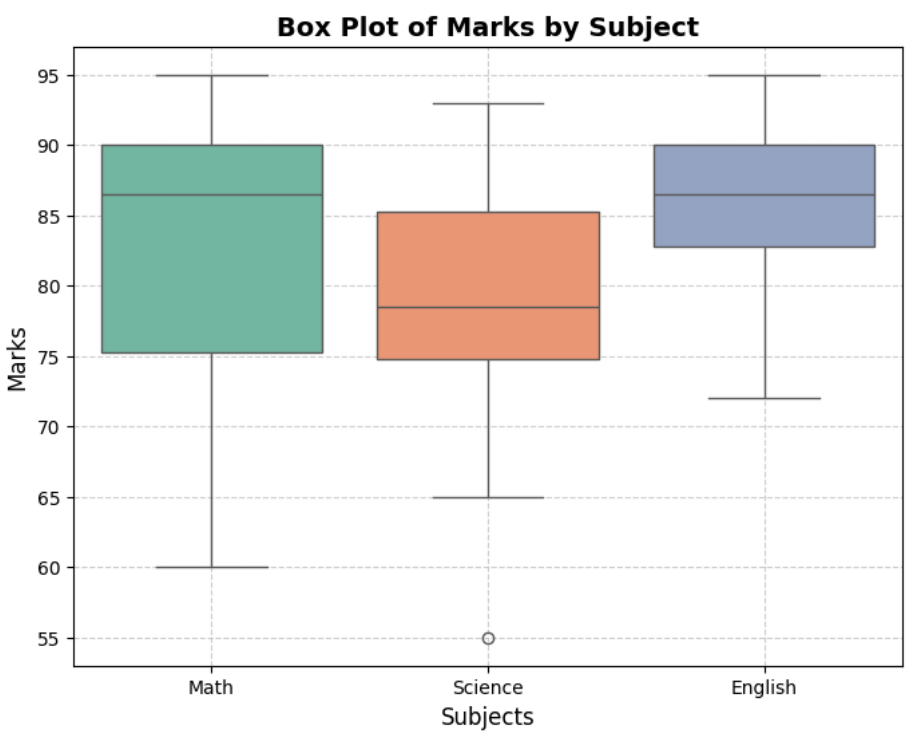
11. Show how to create advanced visualizations such as a box plot and heatmap using Seaborn. Provide examples with code. Explain how to customize these Seaborn plots by adding titles, labels, changing color palettes, and enhancing the visual appeal

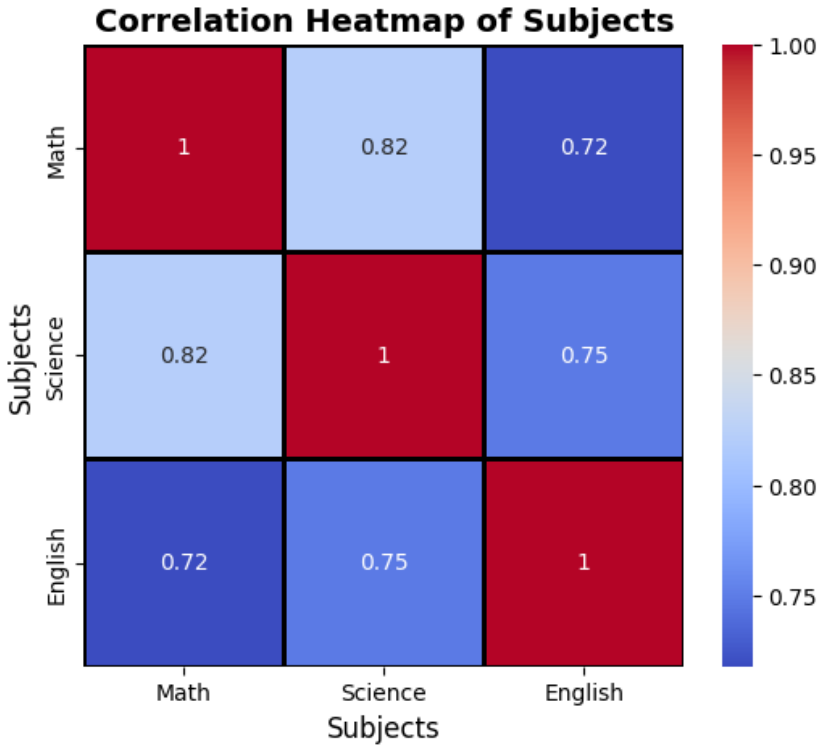
PROGRAM:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# ------------------------------
# Sample dataset
# ------------------------------
data = {
    "Subject": ["Math", "Science", "English", "Math", "Science", "English"] * 10,
    "Marks": [85, 78, 92, 88, 74, 95, 70, 65, 80, 90, 85, 88,
              60, 55, 72, 95, 89, 91, 76, 82, 84, 67, 73, 88,
              92, 79, 85, 81, 77, 90, 68, 72, 94, 88, 75, 83,
              91, 86, 79, 82, 88, 90, 73, 85, 77, 89, 92, 84,
              80, 76, 88, 91, 85, 87, 90, 78, 82, 89, 93, 86]
}
df = pd.DataFrame(data)
```

```
# ------------------------------
# 1. Box Plot
# ------------------------------
plt.figure(figsize=(8,6))
sns.boxplot(x="Subject", y="Marks", data=df, palette="Set2")

# Customization
plt.title("Box Plot of Marks by Subject", fontsize=14, fontweight="bold")
plt.xlabel("Subjects", fontsize=12)
plt.ylabel("Marks", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()

#
```

```python
# 2. Heatmap
# ------------------------------
# Create a correlation matrix using a sample marks dataset
marks_data = {
    "Math": [85, 88, 70, 90, 60, 95, 76, 84],
    "Science": [78, 74, 65, 85, 55, 89, 82, 67],
    "English": [92, 95, 80, 88, 72, 91, 84, 73]
}
df_marks = pd.DataFrame(marks_data)

# Compute correlation
corr = df_marks.corr()

plt.figure(figsize=(6,5))
sns.heatmap(corr, annot=True, cmap="coolwarm", linewidths=1, linecolor="black")

# Customization
plt.title("Correlation Heatmap of Subjects", fontsize=14, fontweight="bold")
plt.xlabel("Subjects", fontsize=12)
plt.ylabel("Subjects", fontsize=12)
plt.show()
```





12. Write a Python program that demonstrates key concepts from the following topics: Python data structures, functions, and data analysis using libraries like NumPy, Pandas, and Matplotlib. The program should:

a) Define a function called analyze_sales_data() that accepts a list of sales figures (integers) and performs the following tasks:
   - **Calculate and return the total, maximum, minimum, and average sales from the list.**

b) Create a NumPy array from the given sales figures and perform basic statistical operations on the array (sum, mean, median).

c) Convert the NumPy array into a Pandas DataFrame and create an additional column for "Region" which is randomly chosen from ['North', 'South', 'East', 'West'].

d) Plot a bar chart of total sales per region using Matplotlib.

Demonstrate basic error handling by checking if the input list is empty and raising an appropriate exception.

Answer:

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random
```

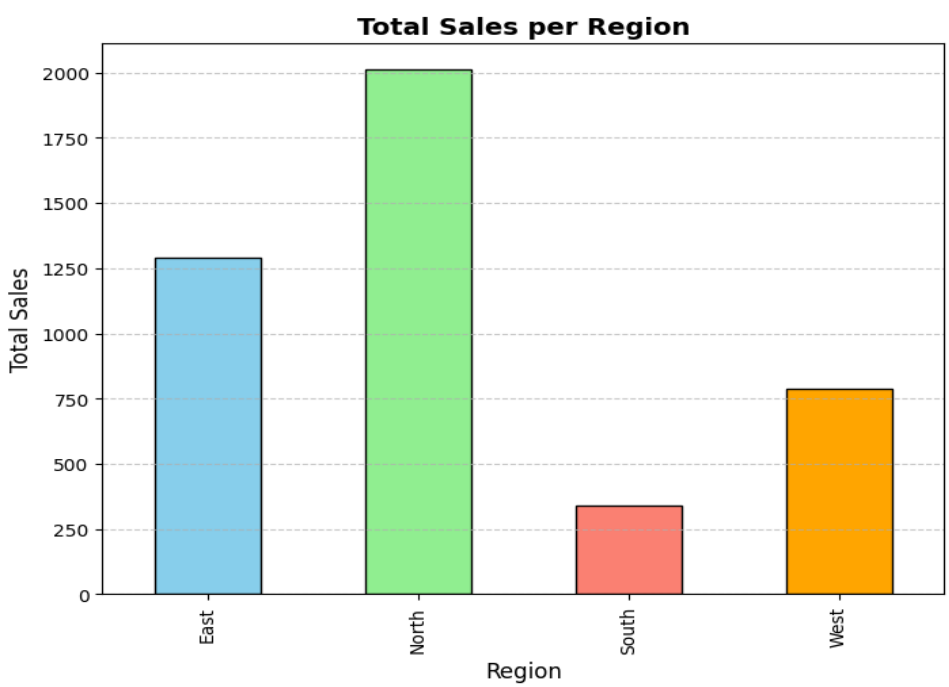
```python
# ----------------------------------
# a) Define function for sales analysis
# ----------------------------------
def analyze_sales_data(sales_list):
    """
    Analyze sales data: total, max, min, average.
    Raises ValueError if list is empty.
    """
    if not sales_list:  # Error handling
        raise ValueError("Sales list cannot be empty!")

    total = sum(sales_list)
    maximum = max(sales_list)
    minimum = min(sales_list)
    average = total / len(sales_list)

    return {
        "Total Sales": total,
        "Maximum Sale": maximum,
        "Minimum Sale": minimum,
        "Average Sale": average
    }


# ----------------------------------
# Example sales data
# ----------------------------------
sales_figures = [120, 340, 560, 230, 450, 670, 890, 150, 300, 720]

# Call the function
results = analyze_sales_data(sales_figures)
print("Sales Analysis Results:")
for key, value in results.items():
    print(f"{key}: {value}")
```

```python
# ----------------------------------
# b) NumPy array operations
# ----------------------------------
sales_array = np.array(sales_figures)

print("\nNumPy Array Statistics:")
print("Sum:", np.sum(sales_array))
print("Mean:", np.mean(sales_array))
print("Median:", np.median(sales_array))

# ----------------------------------
# c) Convert to Pandas DataFrame with Region column
# ----------------------------------
regions = ['North', 'South', 'East', 'West']
df = pd.DataFrame({
    "Sales": sales_array,
    "Region": [random.choice(regions) for _ in range(len(sales_array))]
})

print("\nDataFrame with Regions:")
print(df)
```

```python
# ----------------------------------
# d) Plot bar chart of total sales per region
# ----------------------------------
region_sales = df.groupby("Region")["Sales"].sum()

plt.figure(figsize=(8,6))
region_sales.plot(kind="bar", color=["skyblue", "lightgreen", "salmon", "orange"], edgecolor="black")

plt.title("Total Sales per Region", fontsize=14, fontweight="bold")
plt.xlabel("Region", fontsize=12)
plt.ylabel("Total Sales", fontsize=12)
plt.grid(axis="y", linestyle="--", alpha=0.7)
plt.show()
```

OUTPUT:

```
Sales Analysis Results:
Total Sales: 4430
Maximum Sale: 890
Minimum Sale: 120
Average Sale: 443.0

NumPy Array Statistics:
Sum: 4430
Mean: 443.0
Median: 395.0

DataFrame with Regions:
   Sales Region
0    120   East
1    340  South
2    560   West
3    230   West
4    450   East
5    670  North
6    890  North
7    150  North
8    300  North
9    720   East
```



Demonstrate basic error handling by checking if the input list is empty and raising an appropriate exception.

```python
def analyze_sales_data(sales_list):
    """
    Analyze sales data: total, max, min, average.
    Raises ValueError if list is empty.
    """
    if not sales_list:  # Error handling
        raise ValueError("Sales list cannot be empty!")

    total = sum(sales_list)
    maximum = max(sales_list)
    minimum = min(sales_list)
    average = total / len(sales_list)

    return {
        "Total Sales": total,
        "Maximum Sale": maximum,
        "Minimum Sale": minimum,
        "Average Sale": average
    }
```

```python
# -------------------------------
# Example usage
# -------------------------------
try:
    # Case 1: Valid list
    sales_figures = [120, 340, 560, 230]
    results = analyze_sales_data(sales_figures)
    print("Results with valid list:", results)

    # Case 2: Empty list (will raise exception)
    empty_sales = []
    results = analyze_sales_data(empty_sales)
    print("Results with empty list:", results)

except ValueError as e:
    print("Error:", e)
```

OUTPUT:

```
Results with valid list: {'Total Sales': 1250, 'Maximum Sale': 560, 'Minimum Sale': 120, 'Average Sale': 312.5}
Error: Sales list cannot be empty!
```

13. Create a dataframe using Python for the given table to analyse the workout sessions in a Fitness center. Perform data preprocessing on the given data. Explain the various ways to handle the empty data, wrong data, wrong format and duplicates.

Explain in detail about exploratory data analysis (EDA) and descriptive statistics.

Duration	Pulse	Maxpulse	Calories
60	110	130	409.1
60	117	145	479
60	103	135	340
45	109	175	282.4
60	98	124	269
60		147	329.3
60	100	120	
240	106	128	345.3
60	104	132	379.3

Apply these various commands on the given dataset and plot the analysis results.

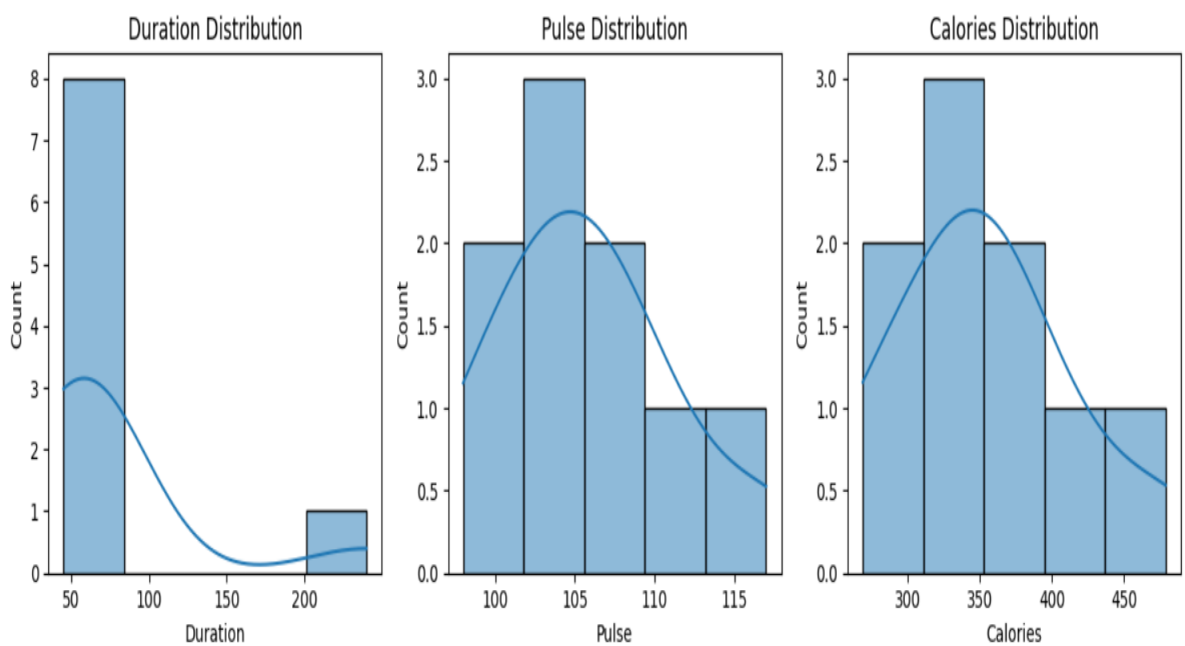
PROGRAM:

```python
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Data from the provided image
data = {
    'Duration': [60, 60, 60, 45, 60, 60, 60, 240, 60],
    'Pulse': [110, 117, 103, 109, 98, np.nan, 100, 106, 104], # Added a NaN to simulate missing Pulse data
    'Maxpulse': [130, 145, 135, 175, 124, 147, 120, 128, 132],
    'Calories': [409.1, 479, 340, 282.4, 269, 329.3, np.nan, 345.3, 379.3] # Added a NaN to simulate missing Calories data
}

df = pd.DataFrame(data)
print("--- Initial DataFrame ---")
print(df)

#1. Handling Empty Data (Missing Values)
# Impute missing 'Pulse' values with the median
median_pulse = df['Pulse'].median()
df['Pulse'].fillna(median_pulse, inplace=True)

# Impute missing 'Calories' values with the mean
mean_calories = df['Calories'].mean()
df['Calories'].fillna(mean_calories, inplace=True)
```

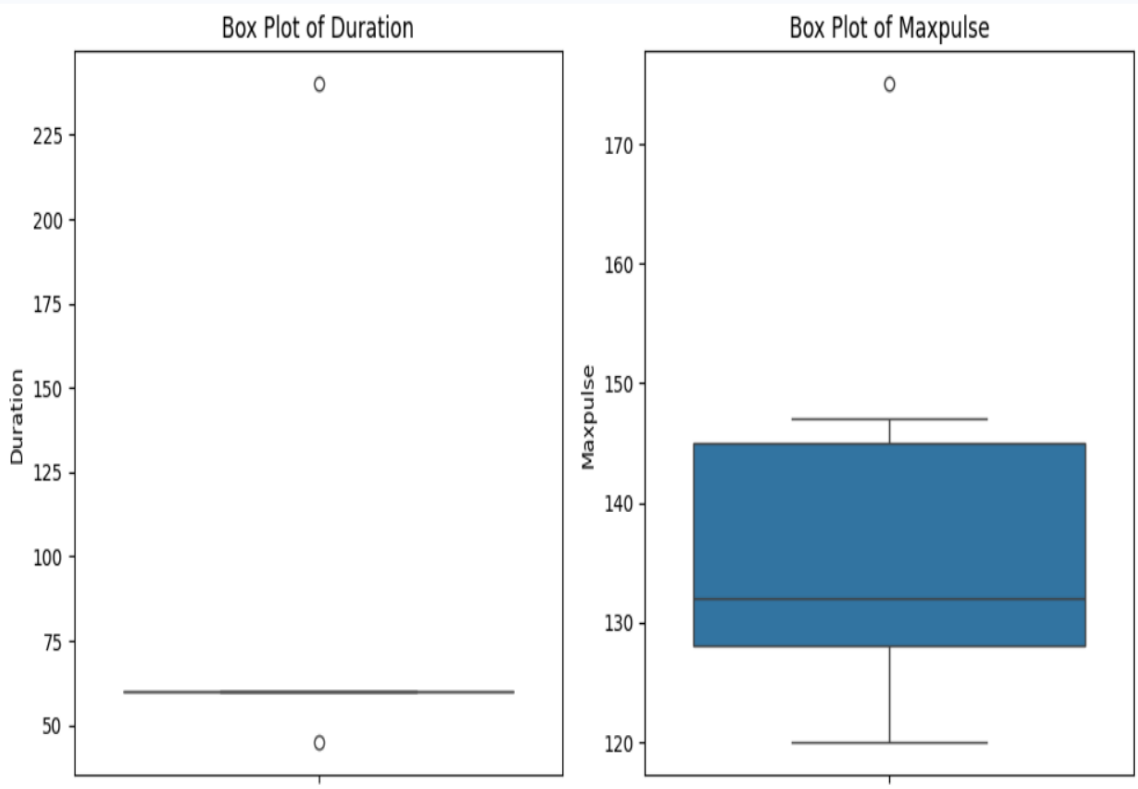
```python
#2. Handling Wrong Data (Inconsistent/Outliers)
# Capping Maxpulse - Replace values above a theoretical maximum (e.g., 180) with a reasonable max.
df.loc[df['Maxpulse'] > 180, 'Maxpulse'] = 180

#3. Handling Wrong Format
# Ensure all columns are numeric
df['Duration'] = df['Duration'].astype(int)
df['Pulse'] = df['Pulse'].astype(int) # After imputation, convert to integer
df['Maxpulse'] = df['Maxpulse'].astype(int)
df['Calories'] = df['Calories'].astype(float)

#4. Handling Duplicates
# Check for and remove duplicates
initial_rows = len(df)
df.drop_duplicates(inplace=True)
removed_duplicates = initial_rows - len(df)
print(f"\n{removed_duplicates} duplicate rows were removed.")
```

```python
#Applying EDA and Descriptive Statistics
#1. Descriptive Statistics Command
print("\n--- Descriptive Statistics of Workout Data ---")
print(df.describe())

#2. Data Distribution Analysis (Histograms)
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
sns.histplot(df['Duration'], kde=True)
plt.title('Duration Distribution')

plt.subplot(1, 3, 2)
sns.histplot(df['Pulse'], kde=True)
plt.title('Pulse Distribution')

plt.subplot(1, 3, 3)
sns.histplot(df['Calories'], kde=True)
plt.title('Calories Distribution')

plt.tight_layout()
plt.show()
```

```python
#3. Outlier Detection (Box Plots)
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
sns.boxplot(y=df['Duration'])
plt.title('Box Plot of Duration')

plt.subplot(1, 2, 2)
sns.boxplot(y=df['Maxpulse'])
plt.title('Box Plot of Maxpulse')

plt.tight_layout()
plt.show()

#4. Relationship Analysis (Scatter Plot and Correlation)
# Calculate the correlation matrix
correlation_matrix = df.corr()
print("\n--- Correlation Matrix ---")
print(correlation_matrix)

# Plot the relationship between Pulse and Calories
plt.figure(figsize=(6, 4))
sns.scatterplot(x='Pulse', y='Calories', data=df)
plt.title('Pulse vs. Calories Burned')
plt.show()
```



```
--- Correlation Matrix ---
          Duration     Pulse  Maxpulse  Calories
Duration  1.000000 -0.003006 -0.276696 -0.016785
Pulse    -0.003006  1.000000  0.508925  0.714401
Maxpulse -0.276696  0.508925  1.000000 -0.149561
Calories -0.016785  0.714401 -0.149561  1.000000
```



Exploratory Data Analysis (EDA) & Descriptive Statistics

What is Exploratory Data Analysis (EDA)?

Exploratory Data Analysis (EDA) is an approach to analyzing data sets to summarize their main characteristics, often with visual methods . It helps discover patterns, spot anomalies, test hypotheses, and check assumptions with the help of statistical summaries and graphical representations.

What are Descriptive Statistics?

Descriptive Statistics quantitatively describe or summarize features of a collection of information. They provide simple summaries about the sample and the measures. Key descriptive statistics include:

- **Measures of Central Tendency:** Mean, Median, Mode.
- **Measures of Dispersion:** Standard Deviation, Variance, Range.
- **Measures of Shape:** Skewness, Kurtosis.